Facial Recognition with Emotion & Liveness

Josh Celestino
104550240

https://github.com/nissanSilviaa/Facial-Recognition

Abstract

This report presents a complete face attendance system combining face verification, anti spoofing, and emotion detection. We describe two embedding approaches, softmax based classification and triplet loss metric learning trained on 1,200 identities, integrate the Laplacian variance method for anti-spoofing, and leverage Meta's deepface CNN for emotion recognition. We evaluate each component quantitatively (accuracy,ROC and AUC, spoof detection rates) and demonstrate that the metric-learning pipeline provides the best overall verification performance.

Table of Contents

Abstract **1**
Table of Contents **1**
Introduction **2**
Methodology **2**
System Framework 2
Results & Discussion **3**
Model Accuracy: 3
Classification Embedding Model 4
Data & Sampling 4
Data Augmentation 4
Model Architecture 4
Hyperparameters 4
Triplet Loss Model 5
References **6**

Introduction

The system must register or recognize faces, reject spoofed inputs, and display emotions. Key components are:

- Face Verification via two embedding strategies (softmax and triplet).
- Anti-Spoofing using texture analysis.
- Emotion Detection via a pretrained FER model.
- GUI for local interaction

Methodology

System Framework

Our system is organized as a linear pipeline of five modules, each handling one key function from raw image to attendance record:

Input & Face Localization

- GUI accepts an image file.
- The system resizes the image down to 64x64 as the model was trained on.

Anti-Spoofing Liveness Check

- Read the full size image (not 64x64) and convert it to grayscale.
- Apply a Laplacian filter and compute its variance.
- If the variance is below a tuned threshold (e.g. 100), the module flags it as a spoof and halt no recognition or emotion analysis.

Face Verification / Embedding

- Read 64x64 image

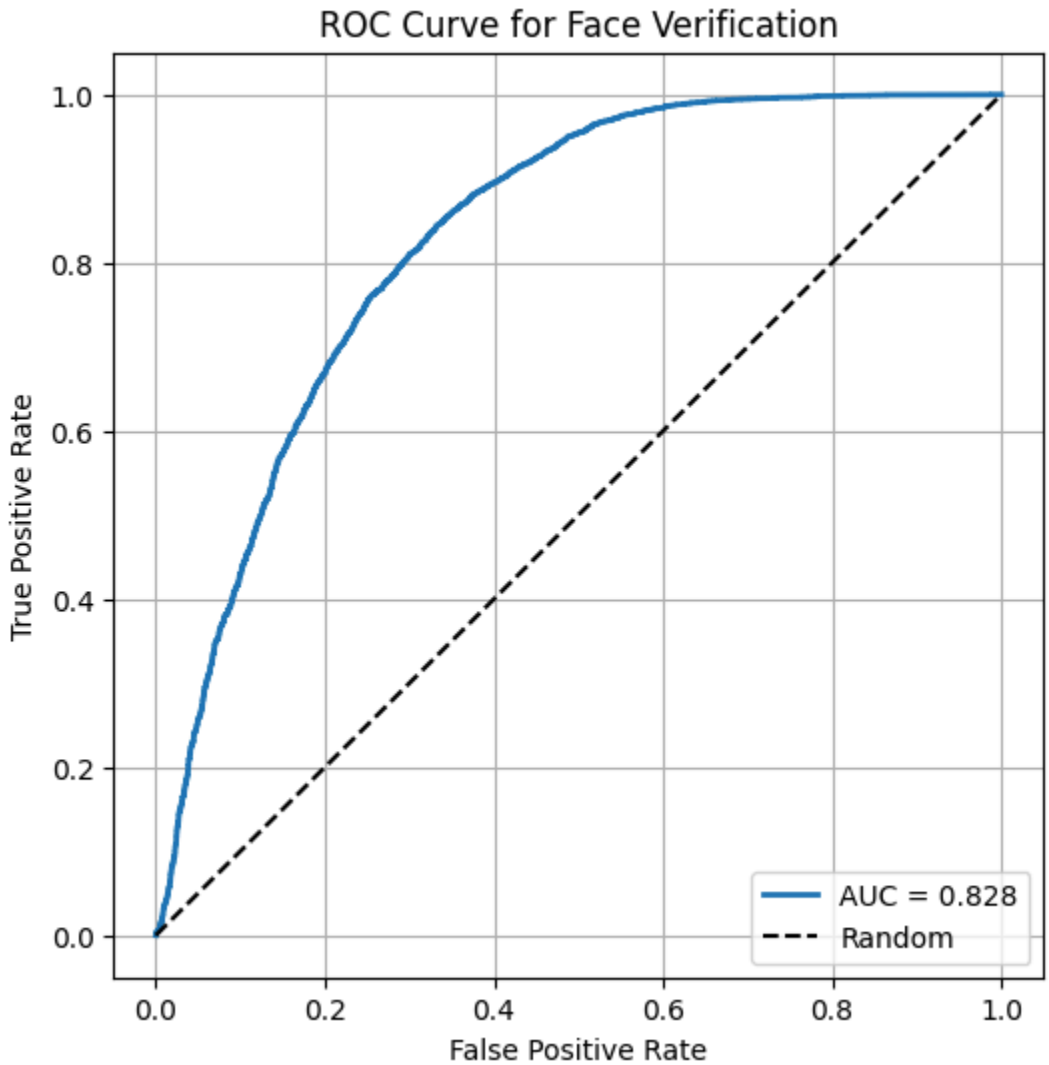
- Pass the live face through our embedding network, which uses a 4 block CNN classification backbone and a small projection head to produce a 128 dim feature vector.
- Projection head: Flatten -> 4096->256 ReLU -> 256->128 embedding
- Compare that vector against our embeddings database (a simple name -> vector dict) using Euclidean distance.
    - If the nearest neighbor distance < threshold, we recognize the existing identity.
    - Otherwise, we prompt the user to register a new name and save its embedding.

Emotion Detection

- Independently send the same face patch into the DeepFace library's pretrained emotion model.
- Extract and display the dominant emotion (happy, sad, angry, etc.) alongside the identity.

Results & Discussion

Model Accuracy:




```
Verification AUC: 0.8285 (82.85%)
```

Classification Embedding Model

Data & Sampling

We list all subfolders in train_data and pick 1,200 identities
Train/Val/Test splits those 1,200 identities/folders

Data Augmentation

```
train_transform = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ToTensor(),
    transforms.Normalize([0.5]*3, [0.5]*3)
])
```

Model Architecture

```
class SimpleCNN(nn.Module):
    def __init__(self, num_classes):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3,32,3,padding=1),  nn.BatchNorm2d(32),  nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32,64,3,padding=1), nn.BatchNorm2d(64),  nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(64,128,3,padding=1),nn.BatchNorm2d(128), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(128,256,3,padding=1),nn.BatchNorm2d(256), nn.ReLU(), nn.MaxPool2d(2)
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Dropout(0.5),
            nn.Linear(256*4*4,512), nn.ReLU(), nn.Dropout(0.5),
            nn.Linear(512,num_classes)
        )
    def forward(self,x):
        x = self.features(x)
        return self.classifier(x)
```

Hyperparameters

Loss Function: CrossEntropyLoss()

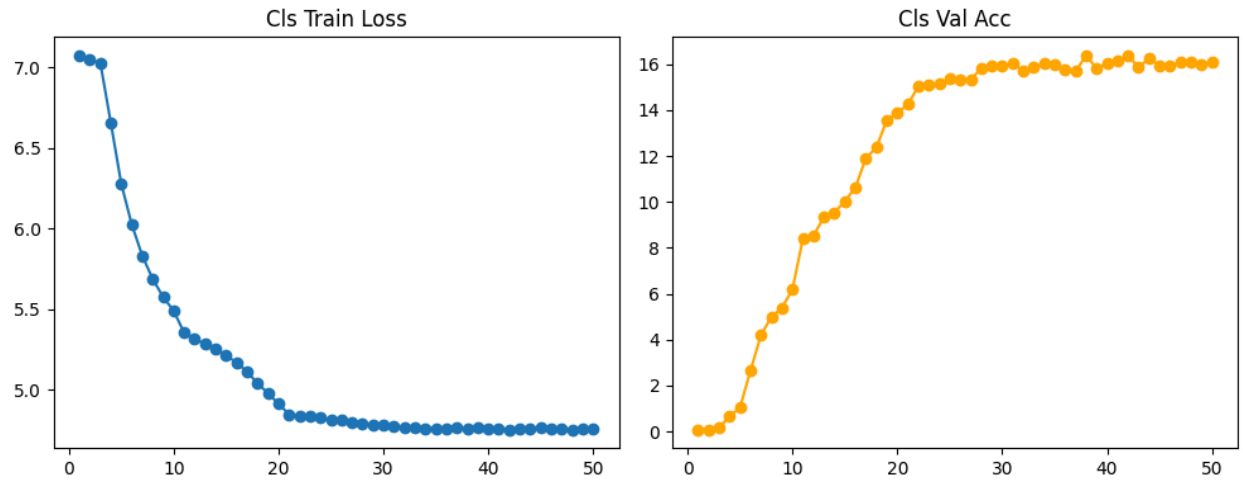
Optimizer: Adam(cls_model.parameters(), learning rate =1e-3, weight_decay=1e-4)

Weight decay = L2 regularization on all weights.

Scheduler: StepLR(optimizer, step_size=10, gamma=0.1)

Batch size: 64

Epochs: 50



Triplet Loss Model

Triplet dataset was trained on the whole **4k identities** and some **360k images** for only **2 whole Epochs (intentionally stopped early)** since I forgot to give it a subnet and resource constraints.
The triplet loss model uses the already trained Classification model's features.

```
embed_net = nn.Sequential(
    cls_model.features,
    nn.Flatten(),
    # 256*4*4 = 4096 features
    nn.Linear(256*4*4, 256),
    nn.ReLU(),
    nn.Linear(256, 128),
).to(device)
```

Loss: TripletMarginLoss()
Optimizer: Adam()
Learning rate: lr=1e-4

Batch size: 32
Epochs: 2 completed (stopped early on the 3rd epoch)

Example of triplet loss training:

References

https://pyimagesearch.com/2015/09/07/blur-detection-with-opencv/
https://github.com/serengil/deepface
https://www.youtube.com/watch?v=kBkwa5Rr8dY&ab_channel=Udacity